E-Commerce Database Management System

ID: 153971

Name: J. Reshma Ann

GitHub URL: https://github.com/reshmaann1998-create/E-Commerce-Database-Management-System

1. Project Overview

- E-Commerce Database Management System is used to manage an online shopping system.
- It stores customer, product, order, cart and payment information.
- Customers can view products, add products to cart and place orders.
- The database keeps all shopping information in an organized way.
- SQL is used to manage and analyze the data.

2. Project Concept

- The project is based on an online shopping system.
- Customers can register and purchase products.
- Products are grouped into different categories.
- Orders and payments are stored in the database.
- SQL queries are used to get useful information from the data.

3. Business Scenario

- Imagine an online shopping company called **ShopEasy**.
- Customers register on the shopping website.
- Customers can view and select products.
- Selected products can be added to the cart.
- Customers can place orders and make payments.
- The company can use the database to check sales and customer information.

4. Features

- Customer management
- Product management
- Shopping cart management
- Order management
- Payment management

5. Technology Used

Technology	Purpose
MySQL	Database management
SQL	Data management and analysis
MySQL Workbench	Creating and running queries
ER Diagram	Showing table relationships

6. Database Tables

No	Table Name	Purpose
1	customers	Stores customer details
2	categories	Stores product categories
3	products	Stores product details
4	cart	Stores customer cart details
5	cart_items	Stores products added to cart
6	orders	Stores order details
7	order_items	Stores products in each order
8	payments	Stores payment details
9	addresses	Stores customer addresses
10	reviews	Stores customer product reviews

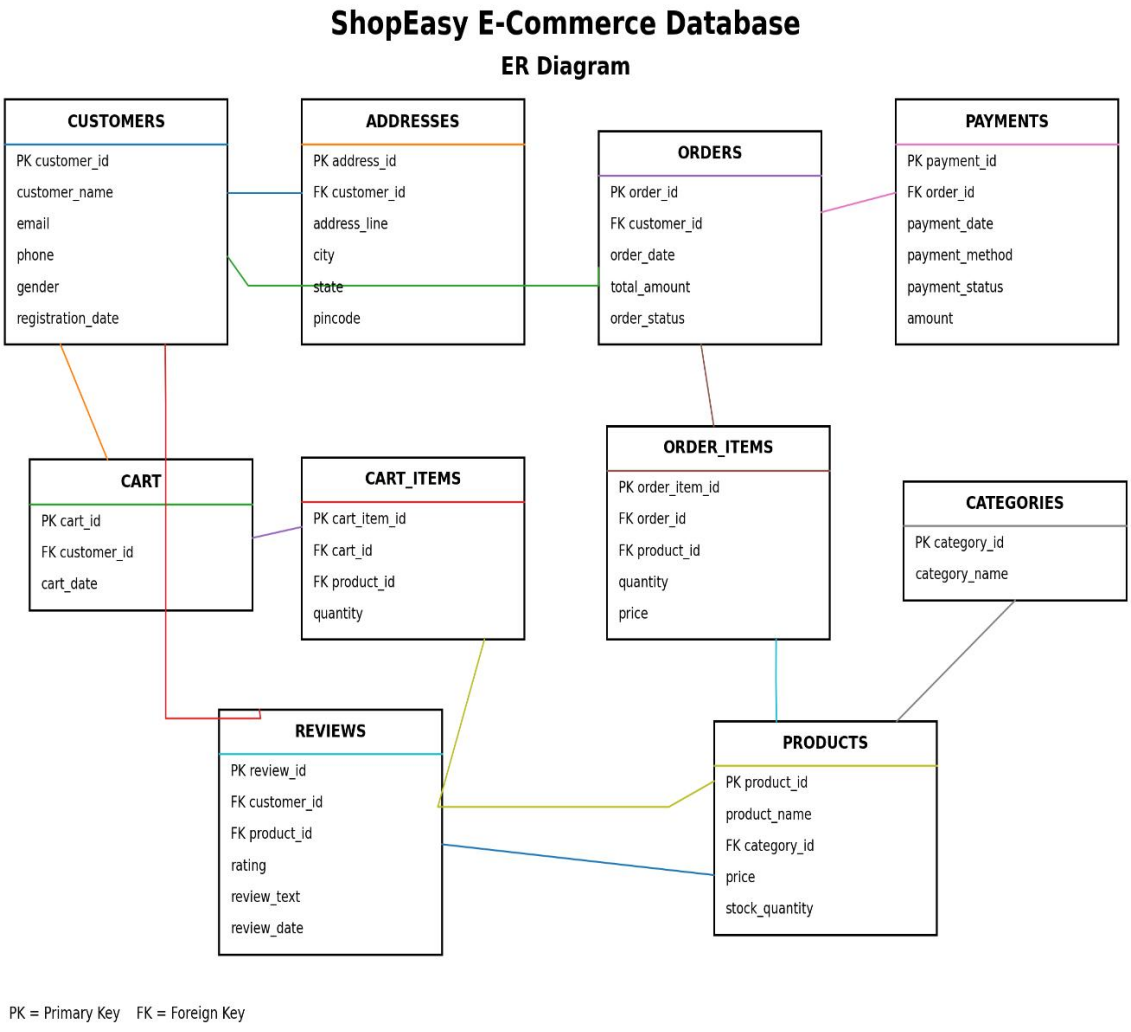
7. Basic Relationships

- One customer can have many orders.
- One customer can have one or more addresses.
- One category can have many products.
- One cart can contain many cart items.
- One order can contain many order items.
- One order can have a payment.
- One product can appear in many orders.
- One product can have many reviews.

8. Entity Relationship Highlights

- customer_id connects customers with orders.
- category_id connects categories with products.
- cart_id connects cart with cart items.
- order_id connects orders with order items and payments.
- product_id connects products with cart items, order items and reviews.

9. ER Diagram



10. SQL Concepts Used

- CREATE DATABASE and CREATE TABLE
- INSERT, UPDATE and DELETE

- SELECT with WHERE and ORDER BY
- INNER JOIN, LEFT JOIN and RIGHT JOIN
- GROUP BY and HAVING
- COUNT, SUM, AVG, MIN and MAX
- Subqueries
- Views
- Window Functions
- RANK () and ROW_NUMBER ()
- LAG () and LEAD ()
- Stored Procedures
- DELIMITER

11. Sample Analytical Queries

1. Find all available products

```
SELECT *
FROM products
WHERE stock_quantity > 0;
```

2. Find customers and their orders

```
SELECT
c.customer_name,
o.order_id,
o.total_amount
FROM customers c
INNER JOIN orders o
ON c.customer_id = o.customer_id;
```

3. Find total sales

```
SELECT SUM (total_amount) AS total_sales
FROM orders;
```

4. Find average order amount

```
SELECT AVG (total_amount) AS average_order
FROM orders;
```

5. Find the most expensive product

```
SELECT *
FROM products
ORDER BY price DESC
LIMIT 1;
```

12. Project Objectives

- To create a simple e-commerce database.
- To store customer and product information.
- To manage carts, orders and payments.
- To connect different tables using relationships.
- To analyze e-commerce data using SQL.

13. Future Enhancements

- Add online delivery tracking.
- Add discount and coupon management.
- Create a sales dashboard using Power BI.
- Add product recommendation features.
- Add sales prediction in the future.

14. Conclusion

- The E-Commerce Database Management System provides an organized way to manage online shopping data.
- It stores customer, product, cart, order and payment information.
- Relationships between tables help maintain accurate data.
- SQL queries help analyze sales and customer information.
- The project provides practical knowledge of database management and SQL.

15. Scenario-Based Questions and SQL Queries

- # Basic Level

1. What are all the products that are currently available in stock?

```
SELECT * FROM products
WHERE stock_quantity > 0
```

```
231    SELECT * FROM products
232    WHERE stock_quantity > 0
```

Result Grid | Filter Rows: | Edit: | Export/Import:

product_id	product_name	category_id	price	stock_quantity
1	Wireless Headphones	1	2500.00	50
2	Smart Watch	1	4500.00	30
3	Bluetooth Speaker	1	3000.00	40
4	Men T Shirt	2	800.00	100
5	Women Dress	2	1500.00	70

products 2 ×

2. Which products have a price `greater` than ₹5,000?

```
SELECT * FROM products
WHERE price > 5000
```

```
233    SELECT * FROM products
234    WHERE price > 5000
```

Result Grid | Filter Rows: | Edit:

product_id	product_name	category_id	price	stock_quantity
13	Smartphone	8	25000.00	20
14	Laptop	9	55000.00	15
NULL	NULL	NULL	NULL	NULL

3.Which products have a price between ₹1,000 and ₹5,000?

```
SELECT * FROM products
WHERE price BETWEEN 1000 AND 5000
```

```
236    SELECT * FROM products
237    WHERE price BETWEEN 1000 AND 5000
```

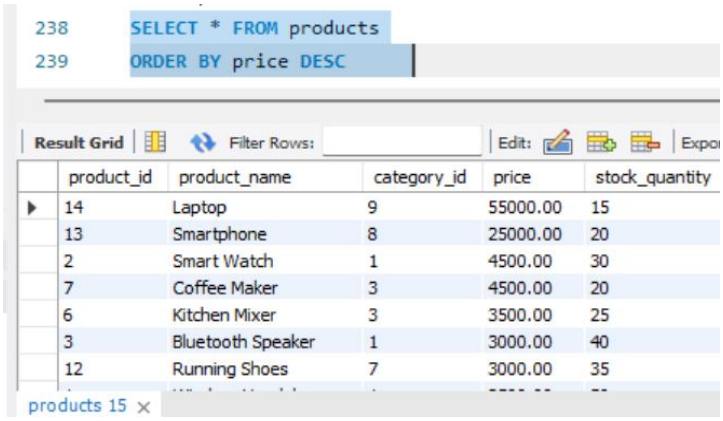
Result Grid | Filter Rows: | Edit: | Expo

product_id	product_name	category_id	price	stock_quantity
1	Wireless Headphones	1	2500.00	50
2	Smart Watch	1	4500.00	30
3	Bluetooth Speaker	1	3000.00	40
5	Women Dress	2	1500.00	70
6	Kitchen Mixer	3	3500.00	25
7	Coffee Maker	3	4500.00	20
11	Sports Shoes	6	2500.00	45

products 14 ×

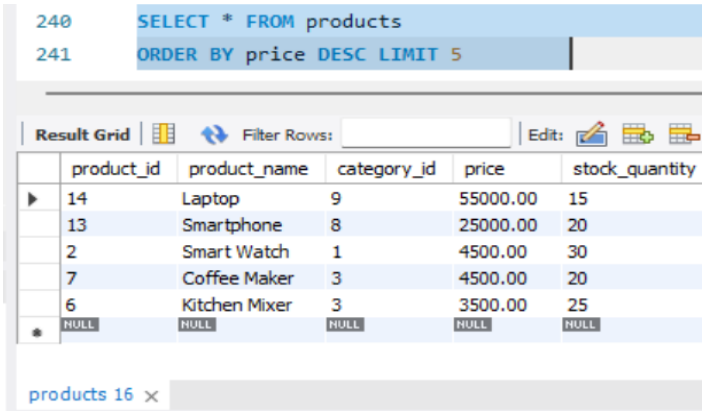
4. Display all products from the highest price to the lowest price.

```
SELECT * FROM products
ORDER BY price DESC
```



5. Show the 5 most expensive products.

```
SELECT * FROM products
ORDER BY price DESC
LIMIT 5
```



6. Find the products whose name contains the word "Book".

```
SELECT *
FROM products
WHERE product_name LIKE '%Book%'
```

```
242  SELECT * FROM products WHERE product_name LIKE '%Book%'
```

Result Grid | Filter Rows: | Edit: | Export/Im

product_id	product_name	category_id	price	stock_quantity
8	SQL Book	4	600.00	50
9	Python Book	4	750.00	40
NULL	NULL	NULL	NULL	NULL

products 17 ×

7. How many customers are registered in the ShopEasy system?

```
SELECT COUNT(*) AS total_customers
FROM customers
```

```
243  SELECT COUNT(*) AS total_customers
244  FROM customers
```

Result Grid | Filter Rows: | Export:

total_customers
10

Result 18 ×

8. How many products are available in the ShopEasy system?

```
SELECT COUNT(*) AS total_products FROM products
```

```
245  SELECT COUNT(*) AS total_products
246  FROM products
```

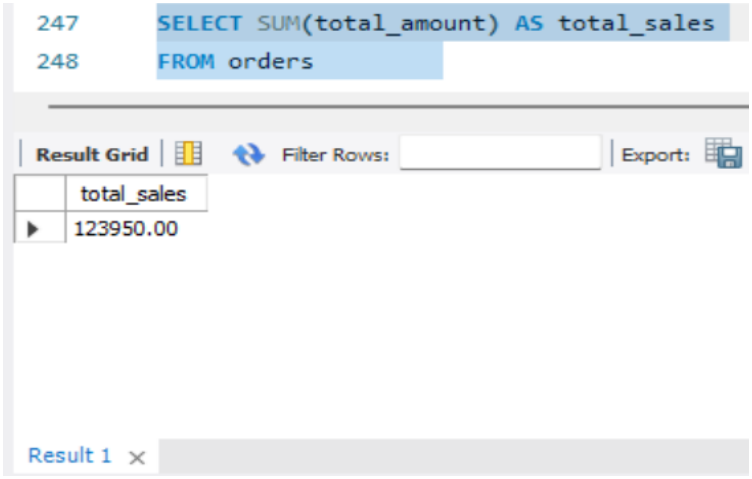
Result Grid | Filter Rows: | Export:

total_products
15

Result 19 ×

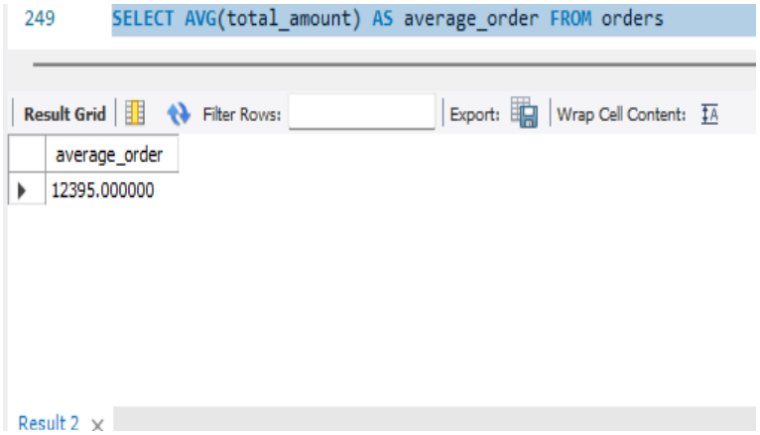
9. What is the total sales amount from all orders?

```
SELECT SUM(total_amount) AS total_sales
FROM orders
```



10. What is the average order amount?

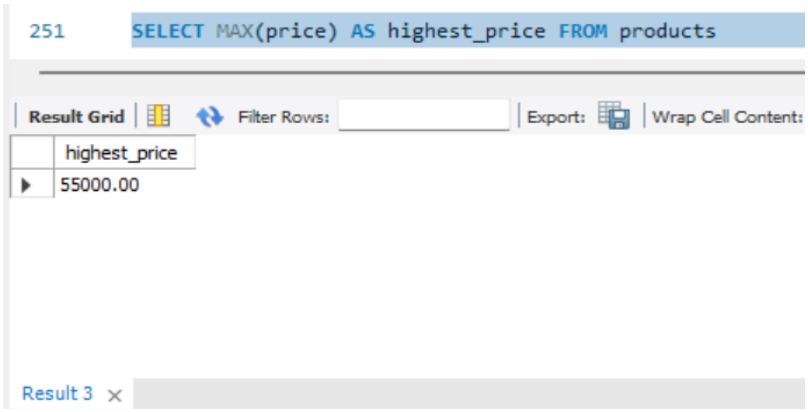
```
SELECT AVG(total_amount) AS average_order FROM orders
```



- Intermediate

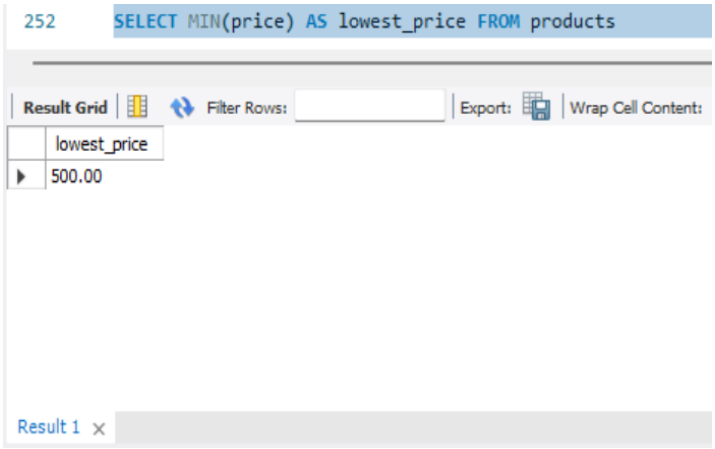
11. What is the highest-priced product in the ShopEasy store?

```
SELECT MAX(price) AS highest_price FROM products
```



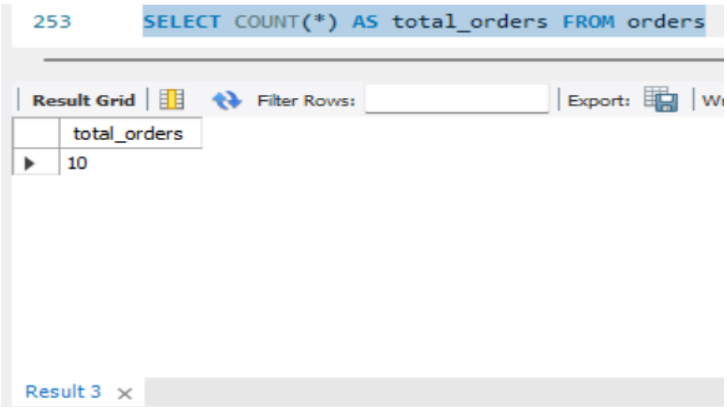
12. What is the lowest-priced product in the ShopEasy store?

```
SELECT MIN(price) AS lowest_price FROM products
```



13. How many orders have been placed by customers?

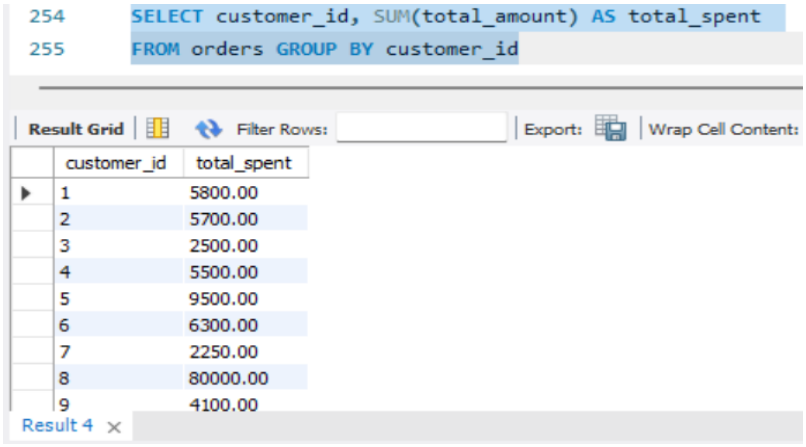
```
SELECT COUNT(*) AS total_orders FROM orders
```



14. Find the total amount of all orders placed by each customer.

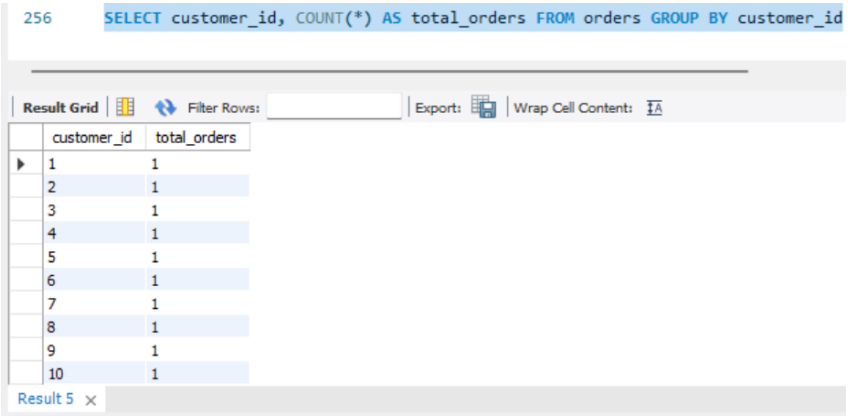
```
SELECT customer_id, SUM(total_amount) AS total_spent
```

```
FROM orders GROUP BY customer_id
```



15. How many orders has each customer placed?

```
SELECT customer_id, COUNT(*) AS total_orders
FROM orders GROUP BY customer_id
```



16. What is the average order amount for each customer?

```
SELECT customer_id, AVG(total_amount) AS average_order_amount
FROM orders GROUP BY customer_id
```

```
257    SELECT customer_id, AVG(total_amount) AS average_order_amount
258    FROM orders GROUP BY customer_id
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	average_order_amount
1	5800.000000
2	5700.000000
3	2500.000000
4	5500.000000
5	9500.000000
6	6300.000000
7	2250.000000
8	80000.000000
9	4100.000000

Result 5

17. Which customers have placed more than one order?

```
SELECT customer_id, COUNT(*) AS total_orders
FROM orders GROUP BY customer_id HAVING COUNT(*) > 1
```

```
259    SELECT customer_id, COUNT(*) AS total_orders
260    FROM orders GROUP BY customer_id HAVING COUNT(*) > 1
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	total_orders

Result 1

18. Show the customers who have placed orders along with their order details.

```
SELECT customers.customer_name, orders.order_id,
orders.total_amount FROM customers INNER JOIN orders ON
customers.customer_id = orders.customer_id
```

```
277  SELECT customers.customer_name, orders.order_id, orders.total_amount
278  FROM customers INNER JOIN orders ON customers.customer_id = orders.customer_id
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_name	order_id	total_amount
Reshma	1	5800.00
Joshua	2	5700.00
Vino	3	2500.00
Samuel	4	5500.00
Priya	5	9500.00
John	6	6300.00
Anitha	7	2250.00
David	8	80000.00
Meena	9	4100.00

Result 1

19. Show all customers and their orders, including customers who have not placed any orders.

```
SELECT customers.customer_name, orders.order_id,
orders.total_amount FROM customers LEFT JOIN orders ON
customers.customer_id = orders.customer_id
```

```
279  SELECT customers.customer_name, orders.order_id, orders.total_amount
280  FROM customers LEFT JOIN orders ON customers.customer_id = orders.customer_id
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_name	order_id	total_amount
Reshma	1	5800.00
Joshua	2	5700.00
Vino	3	2500.00
Samuel	4	5500.00
Priya	5	9500.00
John	6	6300.00
Anitha	7	2250.00
David	8	80000.00
Meena	9	4100.00

Result 3

20. Show all products and their order item details, including products that have not been ordered.

```
SELECT order_items.order_id,products.product_name,
order_items.quantity FROM order_items RIGHT JOIN products ON
order_items.product_id = products.product_id
```

```
281    SELECT order_items.order_id, products.product_name, order_items.quantity
282    FROM order_items RIGHT JOIN products ON order_items.product_id = products.product_id
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

order_id	product_name	quantity
1	Wireless Headphones	2
10	Wireless Headphones	1
2	Smart Watch	1
4	Bluetooth Speaker	1
1	Men T Shirt	1
9	Men T Shirt	2
3	Women Dress	1
5	Kitchen Mixer	1
6	Coffee Maker	1

Result 4

21. Show each product along with its category name.

```
SELECT products.product_name, categories.category_name,
products.price FROM products INNER JOIN categories ON
products.category_id = categories.category_id
```

```
283    SELECT products.product_name, categories.category_name, products.price
284    FROM products INNER JOIN categories ON products.category_id = categories.category_id
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

product_name	category_name	price
Wireless Headphones	Electronics	2500.00
Smart Watch	Electronics	4500.00
Bluetooth Speaker	Electronics	3000.00
Men T Shirt	Fashion	800.00
Women Dress	Fashion	1500.00
Kitchen Mixer	Home Appliances	3500.00
Coffee Maker	Home Appliances	4500.00
SQL Book	Books	600.00
Python Book	Books	750.00

Result 5

22. Show each order along with its payment details.

```
SELECT orders.order_id, orders.total_amount,
payments.payment_method, payments.payment_status FROM orders
INNER JOIN payments ON orders.order_id = payments.order_id
```

```
285    SELECT orders.order_id, orders.total_amount, payments.payment_method, payments.payment_status
286    FROM orders INNER JOIN payments ON orders.order_id = payments.order_id
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

order_id	total_amount	payment_method	payment_status
1	5800.00	UPI	Completed
2	5700.00	Credit Card	Completed
3	2500.00	UPI	Completed
4	5500.00	Debit Card	Pending
5	9500.00	UPI	Completed
6	6300.00	Cash	Completed
7	2250.00	UPI	Completed
8	80000.00	Credit Card	Completed
9	4100.00	UPI	Completed

Result 6

23. Show each customer along with their order and payment details.

```
SELECT customers.customer_name, orders.order_id,
orders.total_amount, payments.payment_method
FROM customers INNER JOIN orders ON
customers.customer_id = orders.customer_id
INNER JOIN payments ON orders.order_id = payments.order_id
```

```
287  SELECT customers.customer_name, orders.order_id, orders.total_amount, payments.payment_method
288  FROM customers
289  INNER JOIN orders ON customers.customer_id = orders.customer_id
290  INNER JOIN payments ON orders.order_id = payments.order_id
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_name	order_id	total_amount	payment_method
Reshma	1	5800.00	UPI
Joshua	2	5700.00	Credit Card
Vino	3	2500.00	UPI
Samuel	4	5500.00	Debit Card
Priya	5	9500.00	UPI
John	6	6300.00	Cash
Anitha	7	2250.00	UPI
David	8	80000.00	Credit Card
Meena	9	4100.00	UPI

Result 7

24. Show each customer and the product IDs they ordered.

```
SELECT customers.customer_name, orders.order_id,
order_items.product_id FROM customers INNER JOIN orders ON
customers.customer_id = orders.customer_id INNER JOIN
order_items ON orders.order_id = order_items.order_id
```

```
291  SELECT customers.customer_name, orders.order_id, order_items.product_id
292  FROM customers INNER JOIN orders ON customers.customer_id = orders.customer_id
293  INNER JOIN order_items ON orders.order_id = order_items.order_id
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_name	order_id	product_id
Reshma	1	1
Reshma	1	4
Joshua	2	2
Joshua	2	8
Vino	3	5
Vino	3	10
Samuel	4	3
Samuel	4	11
Priya	5	6

Result 8

25. Find the total amount spent by each customer.

```
SELECT customers.customer_name, SUM(orders.total_amount) AS
total_spent FROM customers INNER JOIN orders ON
customers.customer_id = orders.customer_id GROUP BY
customers.customer_name
```

```
294    SELECT customers.customer_name, SUM(orders.total_amount) AS total_spent
295    FROM customers INNER JOIN orders ON customers.customer_id = orders.customer_id
296    GROUP BY customers.customer_name
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

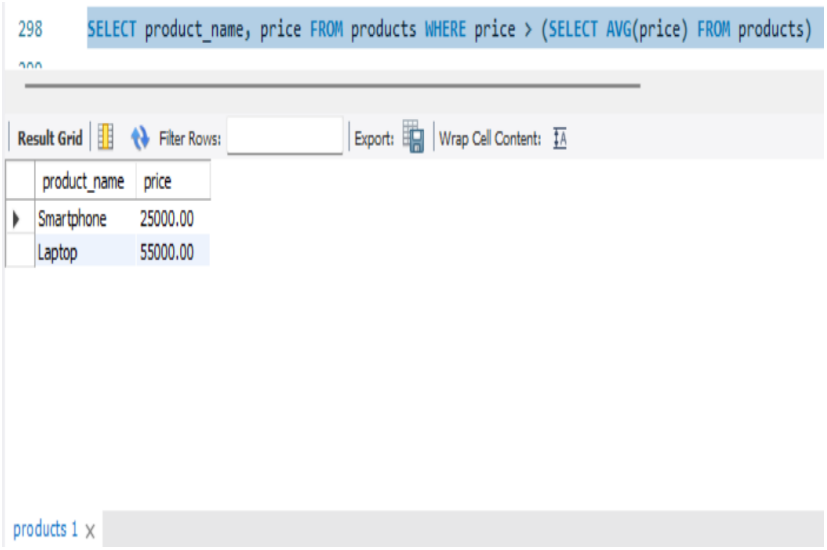
customer_name	total_spent
Reshma	5800.00
Joshua	5700.00
Vino	2500.00
Samuel	5500.00
Priya	9500.00
John	6300.00
Anitha	2250.00
David	80000.00
Meena	4100.00

Result 9

• Advanced

26. Find the products that are priced above the average product price.

```
SELECT product_name, price FROM products WHERE price > (SELECT
AVG(price) FROM products)
```



27. Rank the products based on their price from highest to lowest.

```
SELECT product_name, price, RANK() OVER(ORDER BY price DESC)
AS price_rank FROM products
```

```
299  SELECT product_name, price, RANK() OVER (ORDER BY price DESC) AS price_rank FROM products
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

product_name	price	price_rank
Laptop	55000.00	1
Smartphone	25000.00	2
Smart Watch	4500.00	3
Coffee Maker	4500.00	3
Kitchen Mixer	3500.00	5
Bluetooth Speaker	3000.00	6
Running Shoes	3000.00	6
Wireless Headphones	2500.00	8
Sports Shoes	2500.00	8

Result 2

28. Assign a unique row number to each product based on price from highest to lowest.

```
SELECT product_name, price, ROW_NUMBER() OVER
(ORDER BY price DESC) AS row_num FROM products
```

```
300  SELECT product_name, price, ROW_NUMBER() OVER (ORDER BY price DESC) AS row_num FROM products
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

product_name	price	row_num
Laptop	55000.00	1
Smartphone	25000.00	2
Smart Watch	4500.00	3
Coffee Maker	4500.00	4
Kitchen Mixer	3500.00	5

Result 1

29. Compare each order amount with the previous and next order amount.

```
SELECT order_id, total_amount, LAG(total_amount) OVER (ORDER
BY order_id) AS previous_amount, LEAD(total_amount) OVER
(ORDER BY order_id)
AS next_amount FROM orders
```

```
SELECT order_id, total_amount, LAG(total_amount) OVER (ORDER BY order_id) AS previous_amount,
LEAD(total_amount) OVER (ORDER BY order_id) AS next_amount FROM orders
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

order_id	total_amount	previous_amount	next_amount
1	5800.00	NULL	5700.00
2	5700.00	5800.00	2500.00
3	2500.00	5700.00	5500.00
4	5500.00	2500.00	9500.00
5	9500.00	5500.00	6300.00
6	6300.00	9500.00	2250.00
7	2250.00	6300.00	80000.00
8	80000.00	2250.00	4100.00
9	4100.00	80000.00	2300.00
10	2300.00	4100.00	NULL

Result 4

30. Create a stored procedure to display all products.

```
DELIMITER //

CREATE PROCEDURE show_products()
BEGIN
SELECT * FROM products;
END //

DELIMITER ;

CALL show_products()
```

```
DELIMITER //

CREATE PROCEDURE show_products()
BEGIN
SELECT * FROM products;
END //

DELIMITER ;

CALL show_products()
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

product_id	product_name	category_id	price	stock_quantity
1	Wireless Headphones	1	2500.00	50
2	Smart Watch	1	4500.00	30
3	Bluetooth Speaker	1	3000.00	40
4	Men T Shirt	2	800.00	100
5	Women Dress	2	1500.00	70
6	Kitchen Mixer	3	3500.00	25
7	Coffee Maker	3	4500.00	20

Result 6